

Kubernetes HPA 環境下における コンテナ同居の効果と選択条件

田尾 瑞紀 満田 成紀

複数サービスを1つのPodに配置すること（以下、同居）は通信処理のCPU負荷を下げるが、KubernetesではHPAがDeployment単位で台数を決めるため、同居するとスケールの粒度が粗くなりCPU requests（以下、予約枠）が増える。本稿ではOnline Boutiqueを用いた実機実験により、スケールアウト後も同居の便益が保たれることを確認したうえで、負荷に応じて必要台数の大小が入れ替わる相手では予約枠を調整しても損が残り、通信量の大小だけでは相手を選べないことを示す。

1 はじめに

マイクロサービスのサービス間呼び出しはネットワークスタックを経由し、通信処理のオーバーヘッドが生じる。直接通信するサービスを1つのPodに配置し、通信をlocalhost経由にする同居はこれを削減する。Wickramanayakaら[2]はDocker環境で、通信量の多いコンテナを同一ホストに集め、affinity（サービス間通信のカプセル化込みバイト量）が最大58.5%減少することを示した。この効果を広く利用されるKubernetesに取り入れたいが、中核機能であるHPA（Horizontal Pod Autoscaler、水平方向オートスケーラ）と干渉し、同居するとスケールの粒度が粗くなる。

2 同居とオートスケーラの干渉

HPAはDeployment単位で台数を決めるため、同居したコンテナは同じ台数でしか複製できない。サービス s の予約枠を $r(s)$ （単位millicore）とする。HPA

はCPU使用量を予約枠で割った利用率が目標値に収まるよう台数を増減する。 s を単独のPodで配置したときHPAが到達する台数を s の必要台数 $n(s)$ とする。同居Podの台数は、全コンテナの必要台数を満たす $N = \max_s n(s)$ と定義する（本稿のHPAはコンテナ別利用率を指標とし、実際にも最大値を採る）。このとき $n(s) < N$ のサービスは必要以上に複製され、その予約枠が余分に要求される。その総量

$$\Delta = \sum (N - n(s)) \times r(s) \quad (1)$$

を粒度損と呼ぶ。粒度損は予約枠の余剰であって性能の低下ではなく、必要台数が揃えば $\Delta = 0$ である。

3 実験環境

対象はOnline Boutique[1]（10サービス）で、MicroK8s（単一ノード、16コア）上に置きk6で負荷を与えた。同居は、複数のコンテナを1つのPodに置き、呼び出し元の宛先環境変数をlocalhost:<port>に変えて実現した（アプリケーションの変更はない）。

負荷は2種類（buy・view）の利用者を模した。buyは閲覧・カート追加・購入を1周とし、全10サービスに到達する。viewは閲覧のみを1周とし、frontend・productcatalog（以下catalog）・cart・currency・recommendation（以下reco）・adの6つに到達する一方、checkout・payment・shipping・emailには一切

Effects and Selection Conditions of Container Colocation under Kubernetes HPA

Mizuki Tao, 和歌山大学大学院 システム工学研究科, Graduate School of Systems Engineering, Wakayama University.

Naruki Mitsuda, 和歌山大学戦略情報室, IR Office, Wakayama University.

到達しない。以降、1 秒あたり x 周のレートで与える負荷を $\text{buy}(x) \cdot \text{view}(x)$ と表す。

評価指標は、粒度損が式 (1) の Δ 、通信処理の負荷が softirq の CPU 時間である。先行研究の削減は主にホスト間通信を同一ホスト内に収めてカプセル化を減らしたものであるため[2]、単一ノードではバイト量がほとんど変わらない（実測 +2–4%）。同居で減るのは Pod 間転送のカーネル処理時間であるため、同一ノードの便益は softirq で測る。

3.1 予備実験: スケールアウト後の便益検証

先行研究[2]の効果は台数 1 の比較なので、スケールアウト後の状態を模擬し、HPA を切って台数を 4 に固定した。構成は分離（1 Pod 1 サービス）、 $\text{frontend} \cdot \text{reco} \cdot \text{catalog}$ を同居させた front3 、全サービスを同居させた mega の 3 つで、予約枠は全コンテナ一律（合計 6.3 コア）とした。負荷は基本帯 $\text{buy}(150\text{--}300, 50 \text{ 刻み})$ を 3 サイクル、飽和帯 $\text{buy}(350\text{--}450)$ を 6 サイクル与えた。

基本帯では 3 構成とも目標レートを達成し、1 周あたりの softirq は分離に対し front3 で 17–23%， mega で 43–51% 少なく、台数 4 でも同居の便益は保たれる。飽和帯の $\text{buy}(450)$ では同居が約 440 周/秒に達し、分離（ 359.6 ± 22.9 周/秒）を約 2 割上回った。

3.2 本実験

HPA は目標利用率 70%，台数 1–8，安定化窓を増設 30 秒・削減 60 秒とし、各点は 1 台から開始して 180 秒のウォームアップ後 240 秒を計測した。同居させるサービスの予約枠は実測から決めた。1 台に固定して buy を 6 段階与え、CPU 使用量を負荷の 1 次式で近似してアイドル分 c_0 と 1 周あたり c_1 を求め、予約枠を $(c_0 + 100c_1)/0.7$ とした。 $\text{buy}(100)$ で利用率 70% に揃える設計である。その上で $\text{buy}(100)$ を一定に保ちつつ $\text{view}(0 \cdot 100 \cdot 200 \cdot 300)$ の 4 段階で与え、6 つの組み合わせを測った（5 サイクル）。表 1 に必要台数の食い違いを示す。上段の 3 組は反転せず、下段の 3 組は反転した。

表 1 必要台数の食い違い

組み合わせ	view のレート (周/秒)				計
	0	100	200	300	
$\text{frontend} + \text{reco}$	—	—	1>	2>	3/20
$\text{frontend} + \text{catalog}$	—	—	2>	5>	7/20
$\text{checkout} + \text{email}$	4<	2<	—	—	6/20
$\text{frontend} + \text{checkout}$	1<	2>	2>	4>	9/20
$\text{frontend} + \text{cart}$	5<	—	2>	3>	10/19
$\text{frontend} + \text{email}$	5<	—	2>	4>	11/20

$n <$ は左, $n >$ は右のサービスが n 回引きずられた（相手に合わせて余分に複製された）ことを表す。

4 考察

向きが反転するペアの損は予約枠の調整では抑えられない。各組の全 20 点で損を消せる予約枠を探すと、向きが一定な 3 組は適正值から 7–11% ずらすだけで解が得られた一方、反転する 3 組は 19–42% を要した。必要台数の伸び方が異なるペアでは、負荷の種類や量が変わる度に別の値が必要になる可能性が高い。したがって伸び方が異なるペアは同居相手として選ばないのが妥当である。

一方、同居で取り込む affinity の大小と反転の有無は対応しない。反転した 3 組は $\text{frontend} + \text{cart}$ (affinity 全体の 4.1%)・ $\text{frontend} + \text{checkout}$ (2.5%)・ $\text{frontend} + \text{email}$ (0%) で、 $\text{frontend} + \text{catalog}$ (38.8%)・ $\text{frontend} + \text{reco}$ (4.8%)・ $\text{checkout} + \text{email}$ (1.7%) は向きが一定である。すなわち affinity だけでは相手を選べず、伸び方が揃うかの確認が要る。

5 今後の課題

向きの反転（表 1）は buy と view の比率で生じたため、危険な組み合わせはワークロード構成に依存する。利用者の種類が多いアプリケーションと複数ノードでの検証が今後の課題である。

参考文献

- [1] Google Cloud Platform: Online Boutique v0.10.3, <https://github.com/GoogleCloudPlatform/microservices-demo/releases/tag/v0.10.3>, 2025.
- [2] Wickramanayaka, N. N., Keppitiyagama, C. I., and Thilakarathna, K.: Communication-Affinity Aware Colocation and Merging of Containers, *International Journal on Advances in ICT for Emerging Regions (ICTer)*, Vol. 15, No. 3(2022), pp. 33–45.